Lab 13: Shrink noisy npm test output before Claude reads it

Topic 1: Agentic Architecture & Orchestration · Task 1.5: Apply Agent SDK hooks for tool call interception and data normalization

Runs on Simulator 2: Code Generation with Claude Code · Self-practice

Source: SkillCertPro practice exam Part 1, Q9 (unofficial third-party question)

Commit to an answer first

When Claude Code runs npm test through Bash, the raw stdout often includes long dependency warnings and absolute local paths. The team wants Claude to reason from a shorter, normalized version of successful Bash results while leaving the command execution itself unchanged. What implementation should they choose?

A. Configure a PostToolUse hook matching Bash that exits with code 2 and writes normalized stdout to stderr.

B. Configure a PostToolUse hook matching Bash that returns updated tool output with a Bash result shape.

C. Configure a PostToolUse hook matching Bash that returns decision: "block" with a normalization reason.

D. Configure a PreToolUse hook matching Bash that rewrites stdout inside the incoming tool input.

Simulate the solution

```
# In the browser: open Simulator 2, then Lab 13
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 13

# In a terminal, from simulators/simulator-2-code-generation/
uv run casekit.py --case UC-9
```

What you should see

The command still runs. Claude reads no npm WARN lines and no absolute paths.

The command still runs unchanged, but Claude's reply shows the stripped, path-normalised output: no npm WARN lines, no absolute paths.

Open `simulators/simulator-2-code-generation/evidence/` and find it yourself.

The answer

B. A PostToolUse hook on Bash returns updated output, leaving the command as is.

PostToolUse is the event for transforming a successful tool result after execution but before Claude reasons over it. The hook receives the tool's arguments and response and can return updated output that replaces what Claude sees on the next request. It does not undo the action: the command has already run. For built-in tools such as Bash, the replacement must match the tool's structured output shape. According to the video, a plain string where a structured Bash result is expected may be ignored, leaving the original output in place.

Why the others fail

A. Exit code 2 in PostToolUse shows stderr to Claude as extra feedback. The original output still stands, so this adds text rather than replacing it.

C. decision: "block" sends a reason back to Claude. It does not substitute normalised output.

D. PreToolUse runs before execution, when no stdout exists yet. The tool input is the command, not its output.

Check yourself

- dependency warnings are gone from what Claude read
- absolute local paths are replaced
- the useful test line survived

Now change one thing

Set LAB_MODEL=claude-sonnet-5-5, rerun, and compare the result and the cost line in evidence/.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.